EPL 442 – Machine Learning — Assignment 3: Kohonen Self-Organizing Map

Name: Andreas Evagorou **AM:** 1134609

1. Introduction

The data are the letters of Assignment 2: 20,000 images of the 26 capital letters, made by Frey and Slate (1991) from 20 distorted fonts and reduced to 16 integer features in 0–15 (box size and position, ink moments, edge counts). I keep the same split: the first 16,000 patterns for training, the last 4,000 for testing. The task is to group the letters with a Kohonen Self-Organizing Map (SOM) and measure how well the groups match the real letters.

A SOM is a 2-D grid of neurons, each with a weight vector in the 16-D input space. For every input the closest neuron wins, and the winner and its neighbours *on the grid* move towards the input. Over many passes the grid folds into the data, so neighbouring neurons respond to similar inputs: the 16-D data are drawn as a 2-D map that keeps their topology.

Unlike back propagation in Assignments 1 and 2, there is no teacher: no target, no error to descend. The letter is never seen in training; it is used only afterwards to name each neuron and to score the test set. A good score therefore means the letters really form separate groups in feature space.

2. Solution design and reasoning

Grid. A square 20×20 grid (400 neurons) gives about 15 neurons per letter, enough for each letter to split into sub-groups for its fonts, while each neuron still sees about 40 of the 16,000 patterns. 10×10 (4 per letter) seemed too coarse; Section 4 compares 10×10, 15×15 and 20×20. I chose a square topology because the grid distance is then the plain Euclidean distance between (row, column) positions, as in the lecture's neighbourhood formula.

Normalisation. Each feature is min-max scaled to [0, 1], $x' = (x - min)/(max - min)$, with min and max from the *training set only*; the test set uses the same values and is clipped to [0, 1] (feature 16 is never below 1 in training but is 0 in some test patterns). The Euclidean distance is the whole algorithm, so every feature should have the same range and the same say in choosing the winner.

Initialisation. I implemented the lecture's random weights ("random": uniform in [0, 1)) and "sample", where each neuron starts as a random training pattern; both use the seed `randomSeed`. I chose "sample" because the data fill only a small part of the 16-D cube: before training the quantization error is 0.26 with "sample" against 0.83 with "random".

Algorithm. One epoch presents every training pattern once, in a new random order:

```
normalise data; initialise weights (seeded)
for t = 0 .. T-1:                  # T = maxIterations
    eta, sigma = eta(t), sigma(t)
    for x in shuffled training set:
        c = winner of x            # smallest ||x - w_j||
        for every neuron j:
            w_j += eta * h_cj * (x - w_j)
    QE on training and test set
label neurons; test accuracy
```

Formulas (lecture L6). Winner: smallest $d_j = \Sigma_i (x_i - w_{ij})^2$. Gaussian neighbourhood: $h_{cj}(t) = exp(-\|r_c - r_j\|^2 / 2\sigma^2(t))$, with r the grid position; I preferred it to the bubble function because it fades smoothly instead of cutting off. Update after every pattern: $w_{ij}(t+1) = w_{ij}(t) + \eta(t)\, h_{cj}(t)[x_i - w_{ij}(t)]$. Error (quantization error): $QE = (1/N) \Sigma_p \|x_p - w_{c(p)}\|$, on both sets after every epoch.

Decay of η and σ. The lecture decays the radius exponentially, $\sigma(t) = \sigma_0 \exp(-t/\tau)$, but the learning rate linearly, $\eta(t) = \eta(0)(1 - t/T)$. I decay both exponentially between an initial and a final value:

$$\eta(t) = \eta_0(\eta_T/\eta_0)^{t/(T-1)}, \quad \sigma(t) = \sigma_0(\sigma_T/\sigma_0)^{t/(T-1)}$$

This is the lecture's $exp(-t/\tau)$ with $\tau = (T-1)/ln(\sigma_0/\sigma_T)$, so the last epoch uses exactly the final value. The lecture splits training into a clustering stage with a large η and a fine-tuning stage with a small one. With exponential decay η falls below 0.1 after 41 of 100 epochs, leaving most of the run for fine tuning; the linear form stays above 0.1 until epoch 80, and it reaches η = 0 at t = T, so the last epoch would learn nothing. Decaying both on the same schedule also keeps the neighbourhood from staying wide while the steps are already tiny. Figure 1 matches this: the error is noisy until about epoch 40, smooth after.

Labels. After training each neuron gets the most frequent letter among the training patterns it wins (ties: alphabetically first); a neuron with no patterns gets "–". A test pattern is correct if its winner has its letter, so landing on "–" is wrong.

Difficulties. The assignment's parameter file has only an initial radius, and my first version decayed σ to 1. With a Gaussian, σ = 1 still moves the winner's direct neighbours by $e^{-0.5} \approx 0.61$ of its step, so every neuron stayed an average of its neighbourhood: 55.8% accuracy and a final QE (0.281) worse than the untrained map. I added the parameter `finalRadius` = 0.5 (neighbour weight $e^{-2} \approx 0.14$), which raised accuracy to 67.65%, and `initMethod` (sample | random). Speed was the other problem: each pattern updates all 400 neurons, so the update is vectorised with numpy (about 1.3 s per epoch) and the experiments run four at a time.

3. Implementation

Python 3 with numpy (matplotlib only for plots), one file per role: `parameters.py` (reads `parameters.txt` from the current folder), `data.py` (loading, normalisation), `som.py` (grid, initialisation, winner, neighbourhood, update, decay, QE), `training.py` (epochs), `clustering.py` (labels, accuracy), `results.py` (output files), `plots.py`, `experiments.py` and `main.py`. Over 160 unit tests check each formula against hand-computed values. On a synthetic set of four tight Gaussian clusters on a 5×5 map, 25 epochs bring the QE from 0.25 to 0.022, each cluster gets its own neurons near its centre, and the test accuracy is 100%. Scrambling the training letters leaves the weights and `results.txt` identical, so the labels are never used in training.

4. Experiments and results

Table 1 fixes the baseline so it can be reproduced. Figure 1 plots `results.txt`, since the QE is what the SOM reduces and training and test side by side show whether the map generalises. Figure 2 draws `clustering.txt` as the grid, so neighbouring letters can be read directly. Figure 3 splits the test accuracy by letter, because one overall number hides which letters fail. Tables 2 and 3 change one setting at a time and repeat the baseline with five seeds, to separate real effects from noise.

Parameter	Value	Parameter	Value
Grid (rows × cols)	20 × 20, square	Radius σ (initial → final)	10 → 0.5
numInputs	16	Decay of η and σ	exponential
Epochs (maxIterations)	100	initMethod	sample
Learning rate η (initial → final)	0.5 → 0.01	randomSeed	42

Table 1. Baseline parameters (code/parameters.txt), Gaussian neighbourhood.

The baseline (about 2¼ minutes) ends with training QE 0.2253, testing QE 0.2346 and a **test clustering accuracy of 67.65%** (2,706/4,000); on the training set the same labels give 70.3%. 390 of the 400 neurons get a letter, 10 win no pattern.

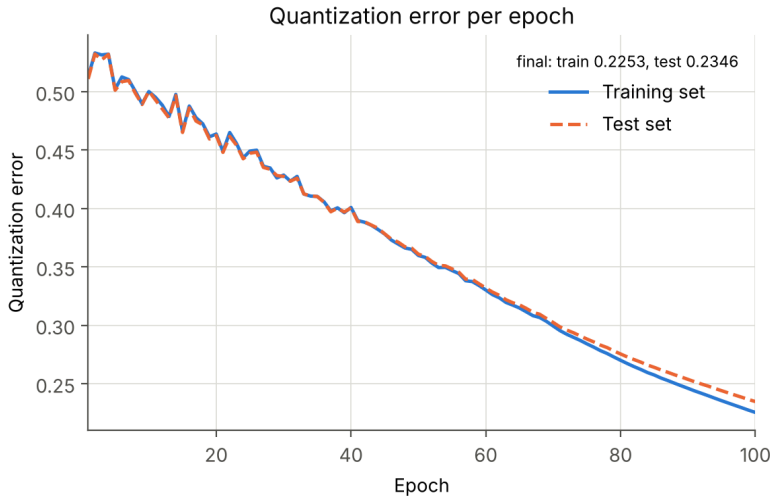


Figure 1. Quantization error per epoch, training and test set (results.txt).

In Figure 1 the QE first *rises*, from 0.26 to 0.53 by epoch 2: with σ = 10 almost the whole map moves with every winner, so the neurons are pulled towards the mean of the data while the map orders itself. It is jagged while η > 0.1 (until about epoch 40), then falls smoothly to 0.2253 and has not levelled off at epoch 100. The two curves stay within 0.01 of each other, so unseen letters are represented as well as the training ones.

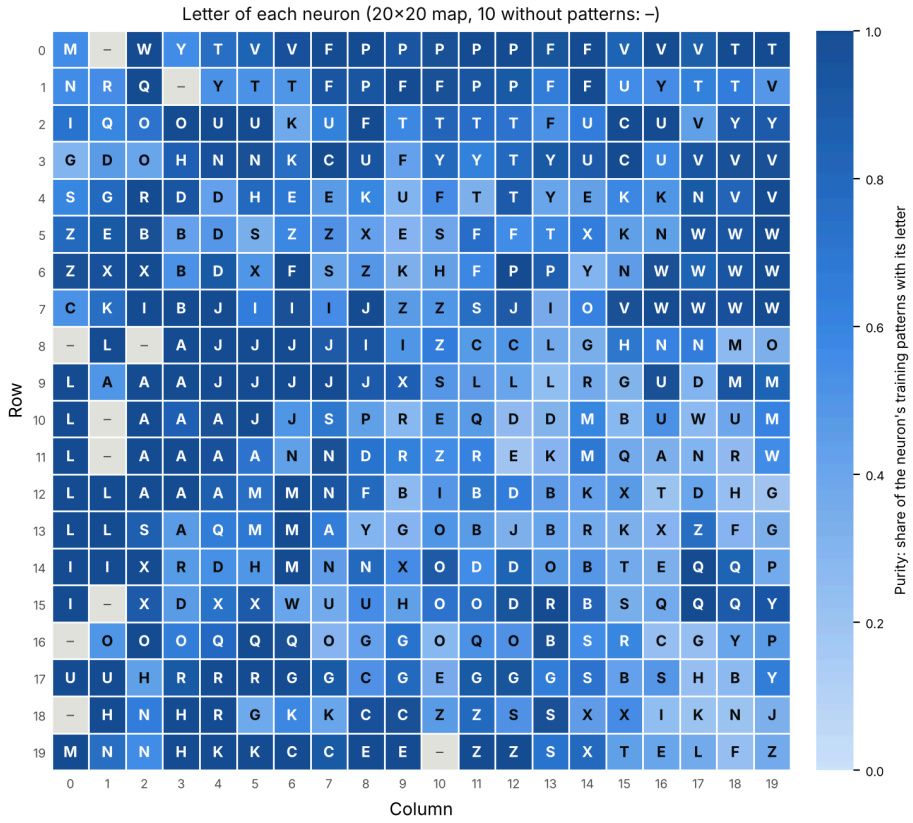


Figure 2. Letter of each neuron (clustering.txt). Shade = purity, the share of the neuron's training patterns with its letter; grey – = no patterns.

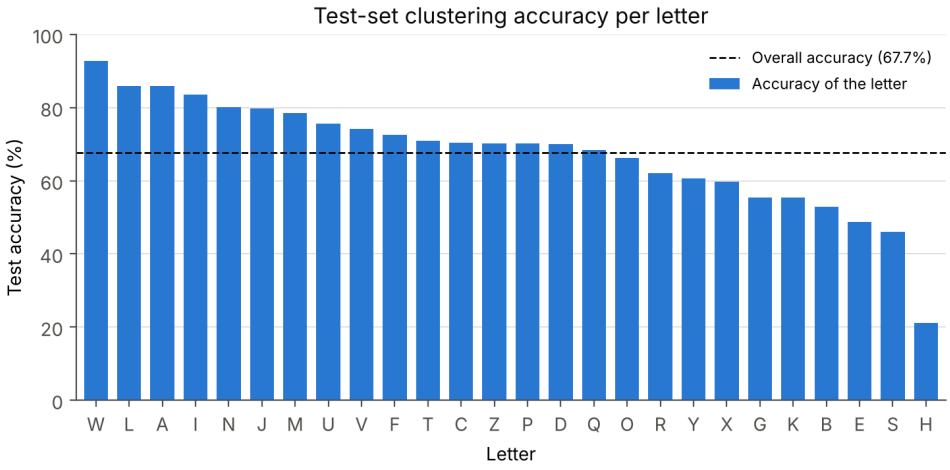


Figure 3. Test-set clustering accuracy per letter (sorted) and overall.

Per letter (Figure 3) the accuracy goes from 92.8% (W), 86.0% (L) and 85.9% (A) down to 48.7% (E), 46.0% (S) and 21.2% (H); 16 of the 26 letters are above the overall rate.

Change from baseline	Train QE	Test QE	Test acc. %	Empty
None (baseline)	0.2253	0.2346	**67.65**	10
Grid 10×10	0.2982	0.3029	46.60	0
Grid 15×15	0.2532	0.2600	58.93	3
η: 0.1 → 0.01	0.2255	0.2343	66.85	10
σ: 10 → 1	0.2806	0.2841	55.80	6
σ: 5 → 0.5	0.2244	0.2340	66.72	9
Random initialisation	0.2248	0.2333	66.95	8

Table 2. Parameter comparisons (100 epochs, seed 42). Empty = neurons without patterns. Full output: code/experiments/summary.txt.

Seed	42	1	2	3	4	Mean ± std
Train QE	0.2253	0.2256	0.2260	0.2253	0.2253	0.2255 ± 0.0003
Test QE	0.2346	0.2339	0.2350	0.2337	0.2347	0.2344 ± 0.0005
Test acc. %	67.65	67.25	66.72	66.38	67.12	**67.03 ± 0.49**

Table 3. The baseline with five seeds (sample standard deviation).

5. Analysis and discussion

The map. Figure 2 has clear regions whose neighbours make sense by shape: a W block on the right with M and N beside it (wide letters, many vertical strokes); P, F and T along the top (ink at the top), with Y and V next to T; the round O, Q, G and C in the bottom middle; the narrow L, I and J on the left next to the A block. The commonest test errors follow these neighbourhoods: H→N (31), S→Z (30), R→B (26), H→K (25), G→O (22) and T↔Y (22/21), letters whose features really overlap. The SOM never sees labels, so it has no reason to draw a border between them. H is the extreme case: its 12 neurons are scattered over the map rather than forming a region, so it reaches only 21.2%.

Empty neurons. 8 of the 10 lie on the left edge between regions (e.g. between the L column and the A block), where the grid stretches across a gap in feature space. They mark separation between groups rather than a fault; only one test pattern landed on one.

Parameters. Grid size matters most: 46.6%, 58.9% and 67.7% for 10×10, 15×15 and 20×20, since 100 neurons cannot hold 20 fonts of 26 letters. Next is the final radius: ending at $\sigma = 1$ costs twelve points (Section 2), while the initial radius, the initial η and the initialisation each change the result by under one point, inside the seed spread of Table 3 (66.4–67.7%). Whatever the start, the wide early neighbourhood pulls the map into the data (in epoch 1 the QE drops from 0.83 to 0.50 with random weights and rises from 0.26 to 0.51 with samples), and every run ends with the same fine tuning at $\eta = 0.01$.

What I would change. Accuracy is still rising with grid size and Figure 1 is still falling at epoch 100, and training (70.3%) and test (67.65%) accuracy are close, so I would try a larger map (25×25 or 30×30) and more epochs, then a hexagonal grid. For reference, the supervised network of Assignment 2 reached 84.2% on the same split.

6. Conclusions

A 20×20 SOM trained without labels puts 67.65% of unseen letters on a neuron of their own letter (67.03 ± 0.49% over five seeds) and places similar shapes next to each other. The end of training matters more than the start (final radius and map size dominate), the error can rise during the ordering stage without anything being wrong, and the SOM's mistakes are the letters whose shapes really overlap.

7. Declaration of AI tool use

I used an AI assistant (Claude) to help write and test the code, run the experiments and word this report. The design choices are mine, every number comes from the submitted code, and I can explain all of it.